

Git & GitHub Quick Notes



Shubham Sarda

•



Feb 22, 2023

•



47 min read

☰ Table of contents

Introduction

- > Version
- > Installation
- > Configuration
- > How To Start Git Repository
- > Common Git Commands

Git Lifecycle

[Show more](#) ▼

Introduction

Git is a version control system that allows developers to keep track of changes made to their code over time. It allows multiple people to work on the same codebase simultaneously and makes it easy to revert to previous versions of the code if something goes wrong.

GitHub is a website that provides hosting for Git repositories. It is a platform where developers can store their code, collaborate with others on open-source projects, and

track issues and bugs. Developers can use GitHub to create their own repositories, or "repos," which are like folders for their code. They can also fork existing repos to make their own copies, and submit pull requests to propose changes to the original repo.

Version

To check the version of Git installed on your system, you can use the following command in your terminal or command prompt:

```
git --version
```

This will display the version number of Git currently installed on your system. For example, the output may look like this:

```
git version 2.39.2
```

This means that Git version 2.39.2 is currently installed on the system.

Installation

To install Git on your system, you can follow these steps:

Windows

1. Go to the Git website: <https://git-scm.com/downloads>
2. Click the Windows link to download the installer.
3. Once the installer is downloaded, double-click it to launch the installer.
4. Follow the prompts in the installer to complete the installation process.

5. After installation is complete, open a command prompt or Git Bash terminal and run `git --version` to verify that Git is installed.

Mac OS

1. Install Xcode Command Line Tools. Open the Terminal app and run the following command: `xcode-select --install`.
2. Install Homebrew package manager by running this command: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
3. Use Homebrew to install Git: `brew install git`.
4. After installation is complete, open a Terminal app and run `git --version` to verify that Git is installed.

Linux

Git is likely available in your distribution's default package repositories, so you can use your package manager to install it. For example, on Ubuntu or Debian-based systems, you can run the following command:

```
sudo apt-get update
sudo apt-get install git
```

After installation is complete, run `git --version` to verify that Git is installed.

Configuration

To configure Git locally, you need to set up a few basic properties that identify you as the

author of the changes you make in Git. Here are the steps to configure Git locally:

1. Set your username: Use the following command to set your Git username:

```
git config --global user.name "Your Name"
```

2. Set your email address: Use the following command to set your email address:

```
git config --global user.email "your-email@example.com"
```

3. Set your preferred text editor: You can set your preferred text editor for Git with the following command:

```
git config --global core.editor "editor-of-your-choice"
```

For example, to set your preferred text editor to Nano, use the following command:

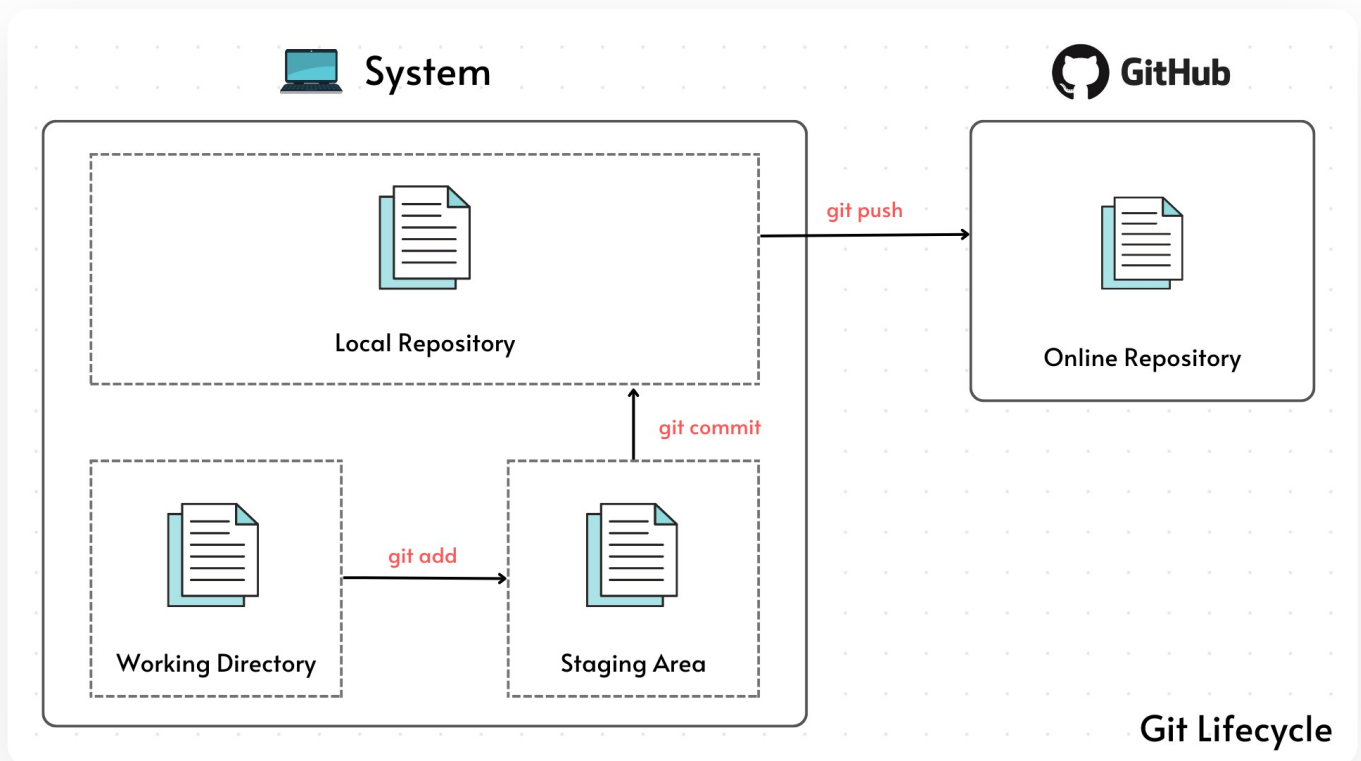
```
git config --global core.editor nano
```

4. Check your configuration: You can check your Git configuration at any time by using the following command:

```
git config --list
```

This will display your Git configuration properties and their values. You can change your Git configuration properties at any time by running the above commands with different values.

How To Start Git Repository



Starting a Git repository and committing changes involves the following steps:

1. Initialize a Git repository in an existing directory:

```
cd <directory>
git init
```

Replace **<directory>** with the name of the directory that you want to turn into a Git repository.

1. Stage and commit your changes:

```
git add <file>
```

```
git commit -m "Your commit message"
```

Replace `<file>` with the name of the file you want to stage and replace `"Your commit message"` with a meaningful message describing your changes.

1. Push the changes to the remote repository using the `git push` command:

```
git push origin <branch>
```

Replace `<branch>` with the name of the branch you want to push the changes to. If you're pushing to the `main` branch, you can use `main` instead of `<branch>`.

Here's a more detailed explanation:

1. Initializing a Git repository: Use the `git init` command in the terminal (or Git Bash on Windows) to turn an existing directory into a Git repository. This will create a new `.git` subdirectory in the directory, which will store all the information about the repository, such as its commit history and configuration.
2. Adding and committing changes: Use the `git add` command to stage changes in your files, and the `git commit` command to commit the changes to the repository's history. The `-m` option allows you to specify a commit message that describes the changes.
3. This will upload your changes to the remote repository, making them available for others to access. Note that you may need to enter your username and password for the hosting service if you're pushing changes to a repository that you don't own.

Note that once you've committed changes to a Git repository, they are stored in the repository's history and can be retrieved at any time using the `git log` command. This makes it easy to revert to previous versions of the code if necessary.

Common Git Commands

git init : This command is used to initialize a new Git repository. It creates a new directory called `.git`, which contains all the necessary files for keeping track of the codebase. This command is typically used when you are starting a new project and want to use Git to track the changes.

git clone : This command is used to create a copy of a remote repository on your local machine. It allows you to download a copy of a repository from a remote location and create a local copy of it. You can then make changes to the local copy and push them back to the remote repository.

git add : This command is used to stage changes for commit. It tells Git that you want to include certain files in the next commit. You can use this command to add individual files or a group of files.

git commit : This command is used to save changes to the repository. It creates a new "commit" that includes all the changes that have been staged with the `git add` command. Each commit includes a message that describes the changes that were made.

git status : This command is used to check the current status of the repository. It shows which files have been modified, which files have been staged for commit, and which branch you are currently on. This command is useful for getting a quick overview of the changes that have been made to the codebase.

git diff : This command is used to show the difference between the current version of a file and the last committed version. It shows the lines that have been added or removed. This command is useful for reviewing changes before committing them.

git log : This command is used to display the commit history of a repository. It shows a list of all the commits that have been made, along with the author, date, and commit

message. This command is useful for reviewing the history of a repository and understanding how it has evolved over time.

git branch : This command is used to create, list, or delete branches in a repository. Branches allow multiple developers to work on the same repository simultaneously without interfering with each other. (The **git branch -M main** command renames the current branch to "main". The **-M** option stands for "move/rename" and is used to move or rename a branch in Git.)

git merge : This command is used to merge changes from one branch to another. It allows developers to combine the changes made in different branches and integrate them into the main branch.

git pull : This command is used to retrieve changes from a remote repository and merge them with the local copy. It fetches changes from the remote repository and then automatically merges them with the current branch.

git push : This command is used to upload changes to a remote repository. It sends changes made in the local repository to the remote repository, updating it with the new commits. (The **git push -u origin main** command pushes the "main" branch to the remote repository named "origin". The **-u** option sets the upstream branch for the current branch.)

- | | |
|-----------------------------------|---------------------------------------|
| ➤ Initialize repository: | git init |
| ➤ Add files to staging area: | git add file_name |
| ➤ Commit to local repository: | git commit -m "first commit" |
| ➤ Rename 'master' branch to main: | git branch -M main |
| ➤ Connect remote repository: | git remote add origin repository_link |



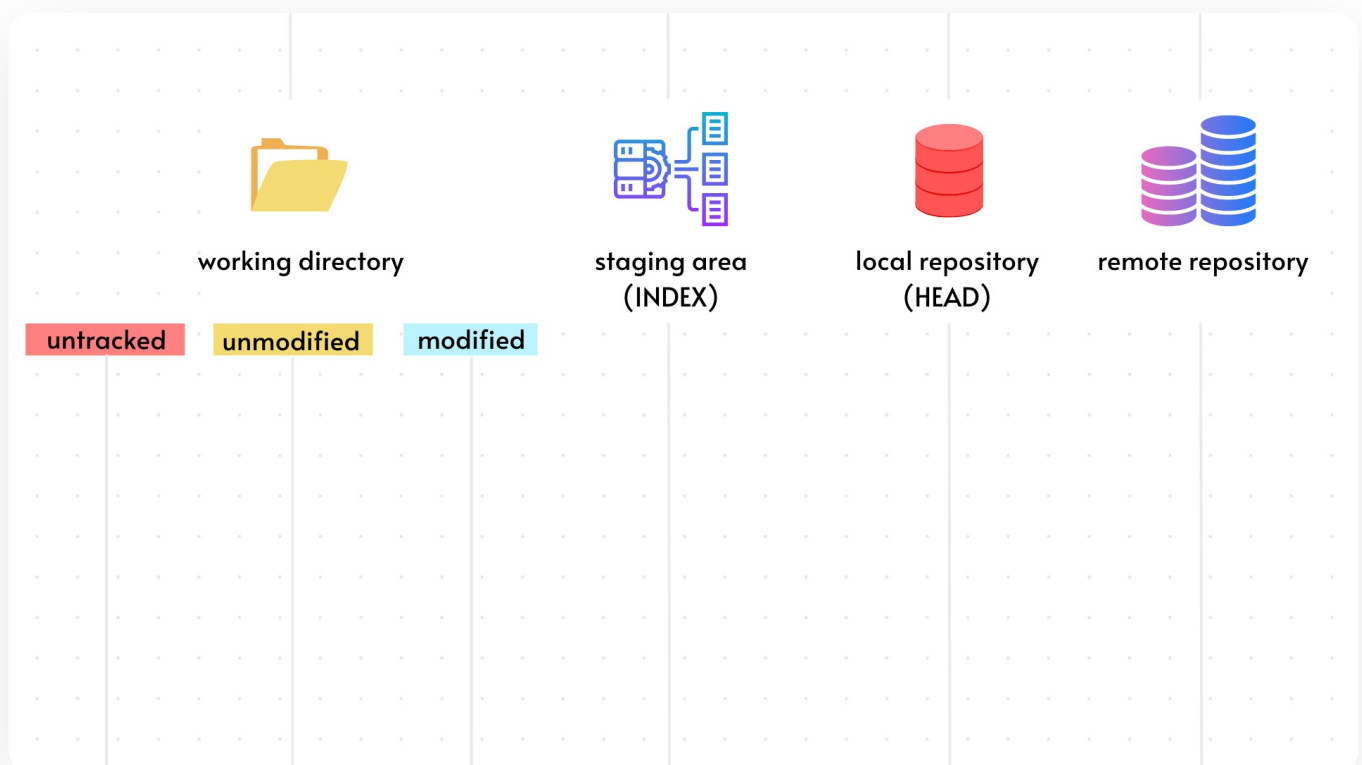
Push all changes:

```
git push -u origin main
```

These are some of the most commonly used Git commands, but there are many other useful commands that you can use to manage your codebase and collaborate with others.

Git Lifecycle

The Git file lifecycle refers to the series of stages that a file in a Git repository goes through as you make changes and commit them.



Here's a simple explanation of the Git file lifecycle:

- **Untracked:** When you first create a file in your Git repository, it is considered

untracked. This means that Git does not yet know about the file, and it will not be included in any commits until you add it to the repository.

- **Staged:** To include a file in a Git commit, you need to stage it. Staging a file adds it to the list of files that will be included in the next commit. You can stage a file using the `git add` command.
- **Committed:** Once you've staged a file, you can commit it to the repository. A commit is a snapshot of your codebase, including all the changes you've made in the staged files. You can make a commit using the `git commit` command.
- **Modified:** After a file has been committed, you can continue to make changes to it. When you make changes to a file, it is considered modified.
- **Staged (again):** To include the new changes in your next commit, you'll need to stage the modified file again. This is because Git only tracks changes that have been staged, not changes made directly to the file.
- **Committed (again):** Once you've staged the modified file, you can make a new commit to save the changes to the repository.
- **Tracked:** When a file has been committed to the repository, it is considered tracked. Tracked files are part of your codebase's history, and changes to them will be recorded in future commits.

The Git file lifecycle is a key part of the Git version control process. By tracking the changes you make to your code and preserving them in commits, Git provides a complete history of your codebase, making it easy to revert changes or compare different versions of your code.

Other Important Commands

Git Diff

The `git diff` command is used to show the differences between two commits, or between the working directory and the most recent commit. Here are some examples of how you might use `git diff` :

1. **Compare working directory and the most recent commit:** To see the differences between the files in the working directory and the most recent commit, you can use the command `git diff` . This will show the changes that you have made but haven't committed yet.
2. **Compare two commits:** To see the differences between two commits, you can use the command `git diff <commit1> <commit2>` . For example, `git diff HEAD~2 HEAD~1` will show the differences between the 2nd and 1st last commits.
3. **Compare branch and another branch:** To see the differences between two branches, you can use the command `git diff <branch1> <branch2>` . For example, `git diff main feature-x` will show the differences between the main branch and feature-x branch.
4. **Compare with a tag:** To see the differences between a commit and a tag, you can use the command `git diff <tag> <commit>` . For example, `git diff v1.0 main` will show the differences between the tag "v1.0" and the main branch.
5. **Viewing specific file:** You can use `git diff <file>` to show the differences between a specific file in the working directory and the most recent commit. For example `git diff index.html` will show the differences between the index.html file in the working directory and the most recent commit.
6. **Color output:** you can add the `--color-words` option to see the specific changes in the words instead of lines, for example `git diff --color-words`

It's important to note that `git diff` command shows the differences between the files, not the entire files. It can be a powerful tool for identifying and resolving conflicts, and for reviewing changes before committing them.

Git Stash

Git stash is a feature in Git that allows you to temporarily save changes you have made in your current branch but are not ready to commit. This is useful when you need to switch to another branch to work on a different task, but don't want to commit the changes you have made in your current branch. Stashing your changes allows you to switch branches and work on the new task without losing the changes you made in your current branch. Later, you can apply the stashed changes back to your current branch or to a different branch.

Here is the basic command to use Git stash:

1. To stash changes in your current branch, run the following command in your terminal or command line:

```
git stash
```

2. To apply the latest stash, run the following command:

```
git stash apply
```

3. To list all stashes, use the following command:

```
git stash list
```

4. To apply a specific stash, use the `stash@{index}` syntax, where index is the stash number from the list. For example:

```
git stash apply stash@{2}
```

5. To drop a stash, use the following command:

```
git stash drop stash@{index}
```

Note that `stash@{0}` is the latest stash, `stash@{1}` is the one before it, and so on. The stash commands are very flexible, and there are many options and variations available. Consult the Git documentation for more information on using Git stash.

Here are some additional Git stash commands:

1. `git stash show` : Display the changes in a stash.
2. `git stash pop` : Apply a stash and remove it from the stash list.
3. `git stash branch <branch-name>` : Create a new branch from the latest stash and apply the stash changes to the new branch.
4. `git stash clear` : Remove all stashes.
5. `git stash drop <stash-name>` : Remove a specific stash from the stash list.
6. `git stash apply --index` : Apply the stash and try to reapply the index changes as well.
7. `git stash create` : Create a stash with a descriptive message.

These commands provide additional options and ways to interact with Git stash, so you can choose the one that fits your needs best. As always, refer to the Git documentation for more information and details on using Git stash.

Git Restore

Git restore is a command in Git that allows you to revert changes in your codebase to a previous version. Here's a simple explanation:

Imagine you've made some changes to your code, but later realize that those changes were a mistake. You want to undo those changes and restore your code to a previous version.

This is where Git restore comes in. With the `git restore` command, you can revert changes in your code to a previous version, effectively undoing the changes you made.

To use `git restore`, you'll need to specify the file or files you want to restore, and the commit that you want to restore them to. For example,

- To restore the contents of a file in the working tree to the version in the most recent commit, use:

```
git restore file.txt
```

- To unstage changes in the index, use:

```
git restore --staged <file>
```

`--staged` allows you to restore the contents in the index.

- To restore the contents of a file to a specific version, use:

```
git restore --source=<commit> <file>
```

`--source` allows you to specify a different version to restore the contents from (default is `HEAD`). You can specify a commit hash, tag, or relative reference (e.g.

`HEAD~1` for the previous commit).

It's important to note that when you restore a file, Git does not delete the changes you made. Instead, it creates a new commit that reverts the changes, allowing you to continue working on your codebase with the previous version of the file.

Git restore is a powerful feature that allows you to undo changes and restore previous versions of your code, making it an essential part of your Git workflow.

Git Branches

Introduction

Branching in Git is a powerful feature that allows developers to work on multiple versions of a codebase simultaneously. It allows developers to create separate branches for different features, bug fixes, or experiments, and then merge them back into the main branch when they are ready.

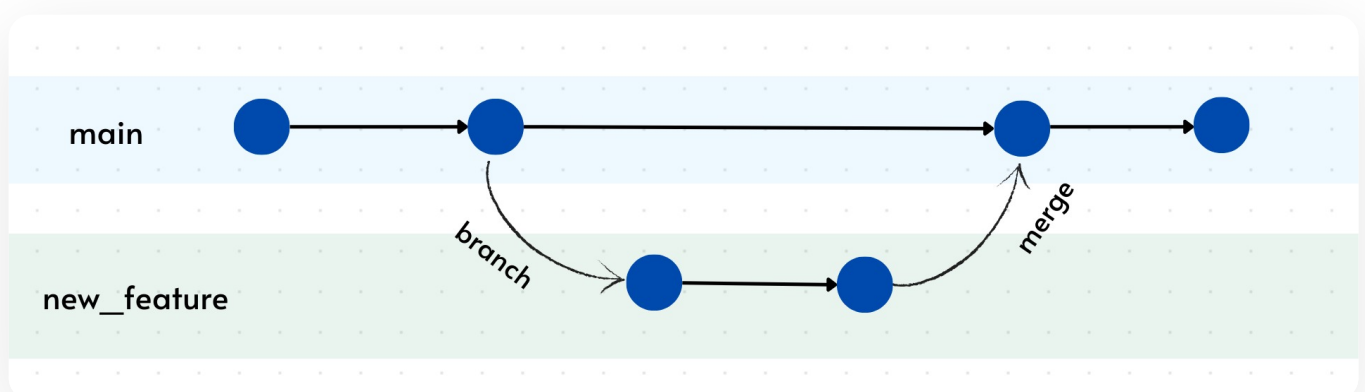
Here is a more detailed explanation of how branching works in Git:

1. **Creating a new branch:** To create a new branch, developers use the `git branch` command, followed by the name of the new branch. For example, `git branch feature-x` will create a new branch called "feature-x". The new branch is created as a copy of the current branch, which is usually the "master" branch.
2. **Switching to a different branch:** Once a new branch has been created, developers can switch to it using the `git checkout` command, followed by the name of the branch. For example, `git checkout feature-x` will switch to the "feature-x" branch. When developers switch to a different branch, their local copy of the codebase is updated to reflect the state of the branch.
3. **Making changes:** Once on a different branch, developers can make changes to the

codebase without affecting the main version of the code or other branches. They can add, modify, or delete files, and then commit the changes using the `git commit` command.

4. **Merging changes:** When a feature or bug fix is complete, developers can merge the changes back into the main branch. To do this, they switch back to the main branch using the `git checkout` command, and then use the `git merge` command to merge the changes from the feature branch.
5. **Deleting a branch:** After merging the changes from a branch back into the main branch, developers can delete the branch using the `git branch -d <branch_name>` command.
6. **Multiple branches:** Developers can create multiple branches, work on them simultaneously and merge them to the main branch.

It's worth noting that Git allows for a non-linear workflow, which means that branches can be merged back into the main branch in any order. This allows for more flexibility and can make it easier to manage large and complex codebases.



Branching is a powerful feature of Git that allows for a flexible and organized development workflow. It allows developers to work on multiple versions of a codebase simultaneously and easily merge changes back into the main branch when they are ready.

Naming Convention

When naming branches in Git, it's important to choose a name that accurately represents the purpose or changes made in that branch. Here are a few tips for naming branches:

1. Use descriptive names: A good branch name should provide clear information about the changes made in that branch. Use meaningful names like "feature-login-page" or "fix-image-resizing-issue".
 2. Use dashes or slashes to separate words: Use hyphens to separate words in a branch name, as it makes it easier to read. For example, instead of using "featureloginpage", use "feature-login-page".
 3. Avoid using special characters and spaces: Special characters can cause problems when working with Git and GitHub, so it's better to avoid them.
 4. Start the name with a feature prefix: If the branch is for a new feature, start the name with "feature-". If it's for a bug fix, start the name with "fix-". This helps other developers quickly understand the purpose of the branch.
 5. Keep branch names short: Long branch names can be difficult to read and can cause confusion. Try to keep branch names short and descriptive.
-

Why Multiple Branches?

Multiple branches in Git allow you to maintain multiple lines of development simultaneously within a single repository. This can be useful for a number of reasons:

1. Isolation of changes: Branches allow you to isolate different changes from each other. This means that you can work on a new feature in one branch, while another branch is still maintaining the existing code.
2. Experimental development: Branches can be used to experiment with new ideas,

features or bug fixes. You can easily switch back to the main branch if the experiment doesn't work out.

3. Collaboration: Branches allow multiple developers to work on different features or bugs simultaneously. When the work is finished, the branches can be merged back into the main branch.
4. Release management: Branches can be used to manage different versions of your codebase. For example, you can use a separate branch to manage the development of a new version while continuing to maintain an older version in the main branch.

In summary, using multiple branches in Git provides a flexible and efficient way to manage changes to a codebase, and can help simplify collaboration and release management.

Git Switch

`git switch` is a command that allows you to switch to another branch in a Git repository. It is an alias for the `git checkout` command and provides a more user-friendly way to switch branches. The basic syntax for `git switch` is:

```
git switch <branch>
```

where `<branch>` is the name of the branch you want to switch to. For example, to switch to the `main` branch, you would run:

```
git switch main
```

This command switches your current branch to the `main` branch and updates your

working directory to reflect the latest version of that branch. After switching to a new branch, you can make changes, stage them, and commit them just as you would with any other branch.

Note that `git switch` was introduced in Git version 2.23 and is available in more recent versions of Git. If you're using an older version of Git, you should use the `git checkout` command instead.

Merge Conflict

A Git merge conflict occurs when two branches have made changes to the same lines in a file, and Git is unable to automatically resolve the differences. This happens when you try to merge one branch into another and the same lines have been modified in both branches.

For example, let's say you have a `main` branch and a `feature` branch. You've been working on the `feature` branch, and you've made some changes to a file called `main.py`. Meanwhile, another developer has been working on the `main` branch, and they've also made some changes to `main.py`. When you try to merge the `feature` branch into `main`, Git will detect that the same lines in `main.py` have been modified in both branches, and it won't know which changes to keep.

When a merge conflict occurs, Git will mark the conflicting lines in the file with special markers that show the changes from each branch. You will need to manually resolve the conflict by deciding which changes to keep, or by combining the changes in a way that makes sense.

To resolve the merge conflict, you can use a Git client that has a built-in merge tool, such as GitKraken, Sourcetree, and others. These tools allow you to visually compare the changes and choose which ones to keep. Alternatively, you can also resolve the conflict using command line.

Once the conflicts are resolved and the changes are made, you will need to commit the changes to finalize the merge.

It's important to note that merge conflicts can be avoided by following a consistent branching strategy, such as using feature branches and pull requests, and by regularly merging changes from the main branch into feature branches. This way, you will minimize the chances of multiple people working on the same files at the same time and creating conflicts.

Resolve Merge Conflicts

Here are the steps to resolve a merge conflict in Git:

1. **Identify the conflict:** When you try to merge one branch into another and a conflict occurs, Git will mark the conflicting lines in the file with special markers. These markers will show the changes from each branch and will look something like this:

```
<<<<<< HEAD
...content from the current branch...
=====
...content from the other branch...
>>>>>> <other_branch>
```

2. **Decide which changes to keep:** Review the conflicting changes and decide which ones you want to keep. You can use a Git client with a built-in merge tool, or edit the files directly in a text editor.
3. **Remove the conflict markers:** Once you've decided which changes to keep, remove the conflict markers and any other unnecessary lines from the file.
4. **Add the modified files:** Use the command `git add` to stage the modified files.

This tells Git that you've resolved the conflicts and the files are ready to be committed.

5. **Commit the changes:** Use the command `git commit` to commit the changes and finalize the merge. It's important to write a clear and concise commit message that describes the changes you've made.
6. **Push the changes:** If you're working on a remote repository, use the command `git push` to upload the changes to the remote repository.

It's important to note that resolving merge conflicts can be a bit tricky, and it may require some experimentation to get the hang of it. And it's always a good idea to communicate with the other developers on the team and coordinate your work to minimize the chances of conflicts happening in the first place.

Pull Before You Push

In Git, it is generally recommended to always run `git pull` before running `git push` if you are working in a shared repository with other contributors.

Here's why:

1. `git pull` updates your local repository with any changes that have been made on the remote repository since your last update. This ensures that you are working with the latest code and that your local changes will be applied cleanly to the current state of the repository.
2. Running `git push` without first running `git pull` can result in conflicts if other contributors have made changes to the same files you have changed. This can cause errors or make it more difficult to merge changes later on.
3. Pulling before pushing also helps to keep the repository history clean and organized, as it ensures that each commit is based on the latest version of the code.

So, to avoid conflicts and keep your repository in sync with the latest changes, it's generally a good practice to always run `git pull` before you run `git push`.

Combined Commands

1. Stage & Commit: `git commit -a -m "message"`
 2. Create Branch & Checkout: `git checkout -b branchname`
-

Git Rebase

`git rebase` is a Git command that allows you to reapply a series of commits from one branch onto another branch. It works by reapplying each commit from the source branch one by one onto the destination branch. This can be used to integrate changes from one branch into another, to clean up a messy commit history, or to simplify a complex branching structure.

Here's a detailed example that demonstrates the use of `git rebase`:

Suppose you have a Git repository with a `main` branch and you've created a `feature` branch to implement a new feature. While working on the `feature` branch, you've made several commits that have added new code to the project. Now you want to integrate your changes from the `feature` branch into the `main` branch.

Here are the steps to do that using `git rebase`:

1. Check out the `feature` branch:

```
git checkout feature
```

1. Rebasing the `feature` branch onto the `main` branch:

```
git rebase main
```

This command reapplies each commit from the `feature` branch onto the `main` branch, one by one. Git will automatically merge each commit into the `main` branch, trying to resolve any conflicts that may arise. If a conflict occurs, you will need to resolve it manually and then continue the rebase by running `git rebase --continue`.

1. Once the rebase is complete, switch back to the `main` branch:

```
git checkout main
```

1. Merge the `feature` branch into the `main` branch:

```
git rebase feature
```

This will create a new merge commit that combines the changes from the `feature` branch into the `main` branch.

With these steps, you've successfully integrated the changes from the `feature` branch into the `main` branch using `git rebase`. This process can be useful for cleaning up a messy commit history, integrating changes from one branch into another, or simplifying a complex branching structure. However, it's important to keep in mind that rebasing can rewrite history, so it should be used with caution, especially if the branch has been shared with other collaborators. In general, it's a good idea to use rebasing only in a private branch that has not been shared with others and to use merging instead when working with shared branches.

Note that rebasing can cause conflicts if the same lines of code have been modified in both branches. In this case, you will need to resolve the conflicts manually before continuing the rebase. Also, rebasing can rewrite history, so it should be used with caution, especially if the branch has been shared with other collaborators. In general, it is a good idea to use rebasing only in a private branch that has not been shared with others, and to use merging instead when working with shared branches.

Difference Between Merge & Rebase

`git rebase` and `git merge` are both Git commands that are used to integrate changes from one branch into another. The main difference between the two is how they integrate the changes.

`git merge` combines changes by creating a new merge commit in the destination branch. This merge commit has two parents: the latest commit in the destination branch, and the latest commit in the source branch. The merge commit represents the combined state of both branches at the time of the merge.

`git rebase`, on the other hand, reapplies the changes from the source branch onto the destination branch one by one, effectively "replaying" the source branch on top of the destination branch. The result is a linear history, with all the changes from the source branch appearing as if they were always part of the destination branch.

Here are a few key differences between `git rebase` and `git merge`:

- History: `git merge` creates a merge commit with two parents, while `git rebase` rewrites the history of the destination branch.
- Conflict resolution: When conflicts arise during a merge, they are recorded in the merge commit. When conflicts arise during a rebase, they must be resolved before the rebase can continue.

- Order of changes: `git merge` preserves the order of changes in both branches, while `git rebase` reorders the changes from the source branch to appear as if they were always part of the destination branch.
- Use case: `git merge` is often used when working with shared branches, as it provides a clear record of the merging of changes from different branches. `git rebase` is typically used when working on a personal branch, to simplify the commit history and make it easier to integrate changes into other branches.

It's important to choose the right command for your needs, as each has its own strengths and weaknesses. If you want to preserve the history of your changes, use `git merge`. If you want to clean up your commit history and simplify your branching structure, use `git rebase`.

Git Files

Git Ignore File

To ignore a file in Git, you need to create a `.gitignore` file in the root of your repository. This file is used to specify files and directories that Git should ignore.

Here are the steps to ignore a file in Git:

1. **Create a .gitignore file:** If a `.gitignore` file doesn't already exist in the root of your repository, create one. You can create the file manually or by using the command `touch .gitignore` in the root of your repository.
2. **Open the .gitignore file:** Open the `.gitignore` file in a text editor.
3. **Add the file to ignore:** Add the name of the file or directory you want to ignore on a new line in the `.gitignore` file. For example, to ignore a file called `temp.txt`, you would add the following line:

```
temp.txt
```

You can also use wildcards to ignore multiple files that match a pattern. For example, to ignore all files with the .log extension, you would add the following line:

```
*.log
```

4. **Save and commit the .gitignore file:** Save the .gitignore file and commit the changes to your repository.
5. **Verify that the file is ignored:** Verify that the file is being ignored by using the command `git status`. The ignored file should not be listed in the output.

It's important to note that ignoring a file only affects the current repository, if you clone the repository or create a new one, you will need to repeat the process to ignore the files again.

Also, if the file you want to ignore has already been tracked by Git, you will need to first remove it from the repository by using the command `git rm --cached <file>` before adding it to the .gitignore file.

Git README File

A README file is a text file that contains information about a project or repository. It typically includes information such as the project's purpose, how to install and use it, and how to contribute to the project. The README file is usually located in the root directory of a repository and is often the first thing that people see when they visit the repository on a code hosting platform like GitHub.

Here are some common elements that you might include in a README file:

1. **Project description:** This should be a brief overview of what the project does and what problem it solves.
2. **Installation instructions:** This should include information on how to install and set up the project, including any dependencies that need to be installed.
3. **Usage instructions:** This should include information on how to use the project, including any command-line arguments or options that can be used.
4. **Examples:** It's a good idea to include some examples of how to use the project, to help users understand how it works.
5. **Contribution guidelines:** If you want other people to contribute to your project, you should include information on how they can do so, including any coding standards or conventions that you follow.
6. **License:** You should include information on the license your project is released under, which tells others what they are allowed to do with your code.
7. **Contact:** If you want to make it easy for users to get in touch with you, you can include contact information such as your email address or social media handles.

It's important to note that the README file should be written in a clear, concise, and easy-to-understand manner and should be kept up-to-date with the latest information about the project.

Other Important Commands

Git Squash

Git squash is a technique that allows you to condense multiple commits into a single commit. This can be useful when you have made multiple commits that are related to a

single change or feature, but you want to present them as a single commit in the repository's history. Squashing commits can make the history of a repository cleaner and more readable.

Here are the steps to squash commits in Git:

1. **Checkout the branch you want to squash:** Make sure you are on the branch that contains the commits you want to squash.
2. **Rebase interactively:** Use the command `git rebase -i HEAD~n`, where "n" is the number of commits from the most recent commit that you want to include in the squash. This will open a text editor with a list of the commits and the word "pick" next to each one.
3. **Change "pick" to "squash":** Change the word "pick" next to the commits you want to squash to "squash". Leave the word "pick" next to the commits you want to keep. Save and exit the text editor.
4. **Edit the commit message:** A new text editor will open, showing the commit messages of the squashed commits. Edit the commit message to include a brief summary of the changes made in the squashed commits. Save and exit the text editor.
5. **Force push:** After squashing the commits, you need to force push the changes to the remote repository using `git push -f <remote> <branch>`.

It's important to note that squashing commits is a destructive operation and can lead to data loss if not done correctly. Before you squash commits, make sure that you have a backup of your work, and that you have coordinated with other members of your team to avoid conflicts.

Git Revert

`git revert` is a Git command that allows you to undo changes in a specific commit and create a new commit to reverse those changes. Unlike `git reset`, `git revert` does not permanently discard changes, but instead adds a new commit that undoes the changes made in a previous commit.

Here is the basic syntax for using `git revert`:

```
git revert <commit>
```

`commit` is the identifier for the commit you want to revert. You can specify a commit hash, tag, or relative reference (e.g. `HEAD~1` for the previous commit).

Example usage:

- To revert the previous commit, use:

```
git revert HEAD~
```

- To revert a specific commit, use:

```
git revert <commit>
```

After running the `git revert` command, Git will prompt you to create a new commit that reverses the changes made in the specified commit. This new commit will be added to the current branch and the changes made in the original commit will no longer be present in the branch.

Note: `git revert` is a safe option when you want to undo changes in a specific commit, as it does not permanently discard changes. Be sure to carefully check the state of your repository before and after using `git revert`.

Git Reset

`git reset` is a powerful Git command that allows you to reset the state of your repository to a specific commit, or unstage changes in the index. It is a versatile command that can be used to undo commits, unstage changes, or reset the current branch to a specific state.

In this blog, we will explore the different types of resets available in Git and how to use `git reset` effectively.

Types of resets in Git

Git provides three types of resets: `--soft`, `--mixed` (default), and `--hard`. The difference between these resets lies in how they affect the repository state.

- `--soft` reset: The `--soft` reset changes the branch pointer to the specified commit, but it does not modify the contents of the working tree or the index. Uncommitted changes in the working tree are left intact.
- `--mixed` reset: The `--mixed` reset, which is the default reset type, changes the branch pointer to the specified commit and resets the index to match the contents of the specified commit. Uncommitted changes in the working tree are left intact.
- `--hard` reset: The `--hard` reset changes the branch pointer to the specified commit, resets the index to match the contents of the specified commit, and discards any changes in the working tree. This type of reset is destructive and should be used with caution.

How to use `git reset`

Here is the basic syntax for using `git reset` :

```
git reset [--soft | --mixed | --hard] [<commit>]
```

<commit> is the identifier for the commit you want to reset to. You can specify a commit hash, tag, or relative reference (e.g. **HEAD~1** for the previous commit).

Here are some examples of how to use **git reset** effectively:

- To unstage changes in the index, use:

```
git reset
```

- To reset the current branch to the most recent commit, use:

```
git reset --hard HEAD
```

- To reset the current branch to a specific commit, use:

```
git reset --hard <commit>
```

- To undo the most recent commit, use:

```
git reset --soft HEAD~
```

- To undo multiple commits, use:

```
git reset --soft HEAD~<number of commits>
```

Note: `git reset` is a powerful command that can be used to undo commits, unstage changes, or reset the current branch to a specific state. However, it is important to be careful when using this command, as it can permanently discard changes. It's a good practice to check the state of your repository before and after using `git reset`.

In conclusion, `git reset` is a versatile command that can be used to reset the state of your repository in various ways. Understanding the different types of resets and how to use `git reset` effectively can greatly simplify your Git workflow and help you to better manage your repository.

Tags and Releases

Git Tag

In Git, a tag is a label that is used to mark a specific point in the repository's history. Tags are typically used to mark a specific version or release of a project and are often used in conjunction with version numbers or release dates.

Here are some examples of how you might use git tags:

- Create a tag:** To create a tag, you can use the command `git tag <tagname>`. For example, `git tag v1.0` will create a tag called "v1.0". You can also add a message to the tag by adding `-a` and `-m` options, for example: `git tag -a v1.0 -m "First official release"`.
- List tags:** To list all the tags in the repository, you can use the command `git tag`. This will display all the tags, in alphabetical order. You can also use `git tag -l` to filter tags by a pattern.
- Push the tag to the remote repository: `git push origin <tagname>`
- View tag details:** To view detailed information about a specific tag, you can use the

command `git show <tagname>` . This will display the tag details, including the changes made in the commit, the author and committer information, and the tag message.

5. **Checkout a specific tag:** To switch to a specific tag, you can use the command `git checkout <tagname>` . This will switch your working directory to the state of the repository at the time the tag was created.
 6. **Delete a tag:** To delete a tag, you can use the command `git tag -d <tagname>`
-

Git Release

Git releases refer to versions of a project that have been officially published and made available to the public. They allow users to download and use a specific version of a project that has been thoroughly tested and considered stable. In a Git repository, releases are typically tagged using Git tags, which mark a specific point in the repository's Git history as being a release. Users can then checkout the repository at that specific tag to access the code and assets for that release. Releases can be used for a variety of purposes, including distributing software, libraries, or other projects to users, or for managing different versions of a project for internal use. The release process generally involves creating a new Git tag, building and testing the code and then publishing the release so that others can access and use it.

Git Clone

`git clone` is a Git command used to create a copy of an existing Git repository (also called "cloning" the repository). It is typically used to download a remote repository from a hosting service like GitHub, GitLab, or Bitbucket to your local machine.

Here's how you can use it:

1. Open the terminal (or Git Bash on Windows) and navigate to the directory where you want to store the local copy of the repository.
2. Run the following command:

```
git clone <repository-url>
```

Replace `<repository-url>` with the URL of the remote repository that you want to clone.

1. The command will download the entire repository and its entire Git history to your local machine. The repository will be stored in a new directory with the same name as the repository.

For example, to clone the official repository of the Ruby programming language, you would run:

```
git clone https://github.com/ruby/ruby.git
```

This would download the entire repository and its entire Git history to a new directory named "ruby" in your current directory.

Recommendation:

To delete the remote repository named "origin" in Git, you can use the following command:

```
git remote rm origin
```

This command removes the remote repository named "origin" from your local Git

configuration. After running this command, you will no longer be able to fetch updates or push changes to this remote repository.

Note that this command only removes the remote repository from your local Git configuration, not from the server where the repository is hosted. If you want to delete the repository from the server, you will need to log in to the server and delete it manually or use the web interface provided by the hosting service.

Open Source Contribution

Introduction

Contributing to open-source projects on GitHub is a great way to learn new skills, give back to the community, and build your portfolio.

Here are the step-by-step commands to contribute to an open-source project on GitHub:

1. **Fork the repository:** Go to the GitHub page of the open-source project you want to contribute to and click the "Fork" button. This creates a copy of the repository under your own GitHub account.
2. **Clone the repository:** Open a terminal and navigate to the directory where you want to clone the repository. Then use the following command to clone the repository to your local machine:

```
git clone https://github.com/<your_username>/<repository_name>.git
```

3. **Create a new branch:** Navigate into the repository directory and create a new branch to work on your changes:

```
cd <repository_name>  
git branch <new_branch_name>
```

4. **Checkout the branch:** Switch to the new branch you just created:

```
git checkout <new_branch_name>
```

5. **Make your changes:** Make the changes you want to contribute to the project.
6. **Add and commit your changes:** Once you've made your changes, you need to add them to the staging area and commit them to your local repository:

```
git add <file_name>  
git commit -m "Your commit message"
```

7. **Push your changes:** Push your changes to your forked repository on GitHub:

```
git push origin <new_branch_name>
```

8. **Create a pull request:** Go to the GitHub page of your forked repository, and click the "Compare & pull request" button. Select the branch you want to submit, add a detailed description of your changes, and submit the pull request.
9. **Discuss and review:** The maintainers of the original repository will review your pull request and may ask you to make some changes or provide additional information.
10. **Merged:** Once your pull request is accepted and merged into the main repository, your changes will be part of the project and you'll become a contributor.

It's important to note that every open-source project has its own guidelines and procedures for contributing, so make sure to read the contributing documentation and

follow the specific instructions provided by the project. Also, it's always a good idea to communicate with the maintainers of the project before making any significant changes.

Pull Request

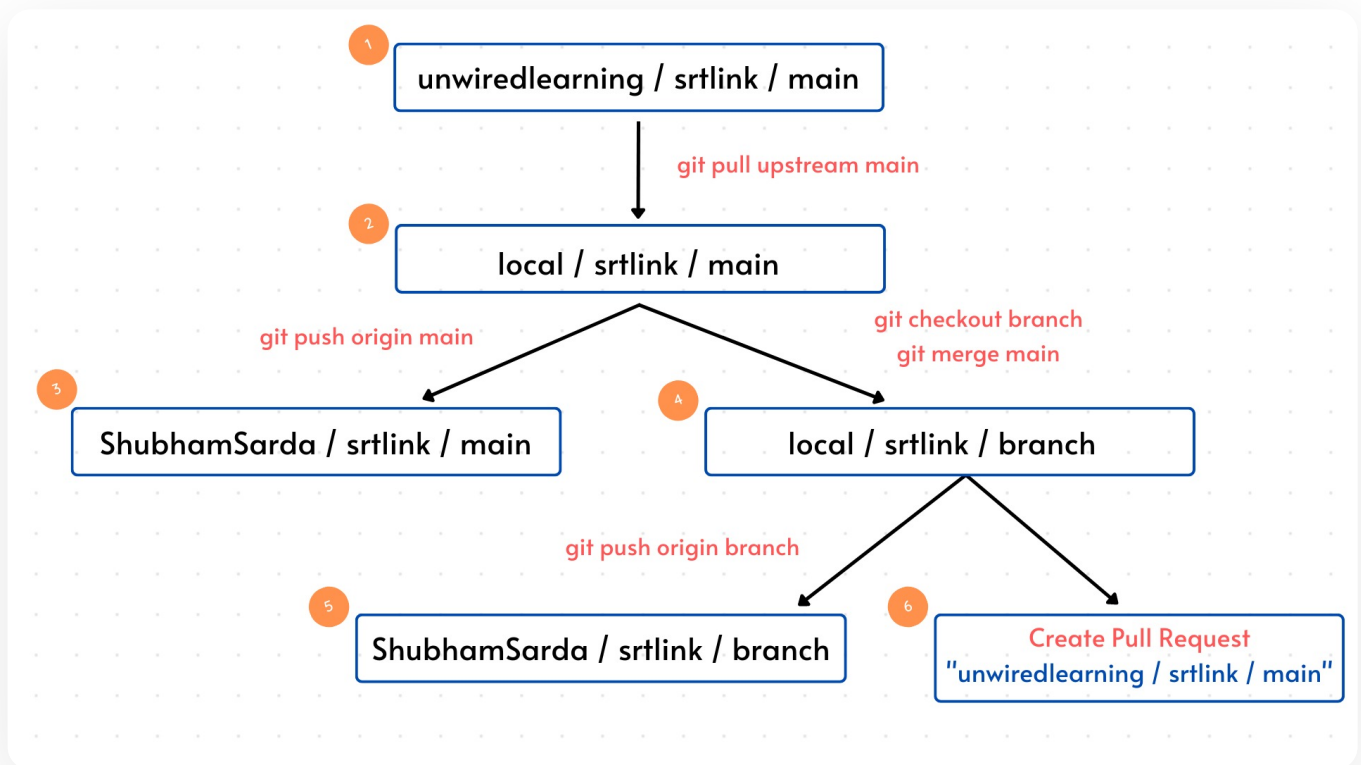
A pull request is a feature in Git and other version control systems that allow developers to propose changes to a codebase and ask for feedback or approval from their colleagues.

Here's how a typical pull request workflow might work:

1. A developer creates a new branch in Git to work on a specific feature or fix.
2. The developer makes changes to the code on their branch, committing changes to the local repository as they go.
3. When the developer is ready to share their changes with the rest of the team, they create a pull request in the repository hosting service (such as GitHub or GitLab).
4. The pull request includes a summary of the changes made, any relevant screenshots or documentation, and a request for feedback or approval.
5. Other team members can review the changes and leave comments, suggest additional changes, or approve the pull request.
6. Once the changes have been reviewed and approved, the pull request can be merged into the main branch, incorporating the changes into the main codebase.

Pull requests are a useful way to collaborate on code and ensure that changes are thoroughly reviewed and tested before they are merged into the main codebase. They can also provide a record of the changes made and the rationale behind them, making it easier to maintain and understand the code over time.

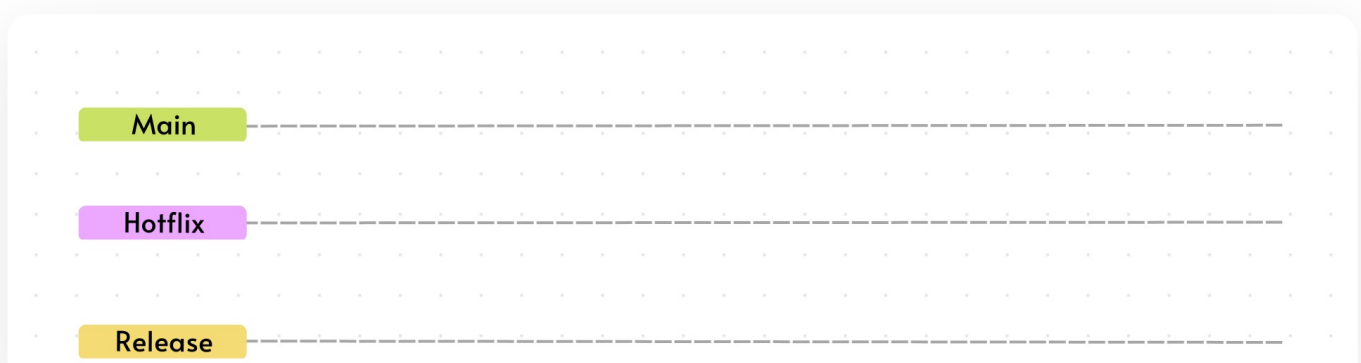
Pull Before You Push



Workflows

Git Workflow

Git flow is a specific branching model that provides a consistent and organized workflow for managing branches and releases in Git. It helps teams to work together effectively by providing a structure for how branches should be created, used, and merged.





Git Workflow

At a high level, Git flow consists of two main branches: `main` and `develop`. The `main` branch is the main branch that always represents the production-ready code, while the `develop` branch is used for ongoing development.

To start a new feature, a new branch is created off of the `develop` branch. This feature branch is used to work on the new feature and make changes to the codebase. Once the feature is complete, it is merged back into the `develop` branch.

When the `develop` branch has a sufficient number of features and it's ready for a release, a new branch is created off of `develop` called `release`. This branch is used to make any final adjustments and fixes before the release.

Once the release is ready, it is merged into both `main` and `develop` branches. The `main` branch is then tagged with a release number and the `develop` branch continues with ongoing development.

Finally, if there are any bugs found in the `main` branch that need to be fixed, a new branch is created off of `main` called `hotfix`. This branch is used to make the necessary changes and once they are completed they are merged back into `main` and `develop` branches.

Git flow provides a clear and consistent workflow that makes it easy for teams to collaborate and work together. It helps to minimize confusion and ensure that the codebase remains stable and ready for production at all times.

Here is an example of how Git flow might be used in a software development project:

1. A development team is working on a new feature for their application, and they want to create a new feature branch to work on it. They use the command `git branch feature-login` to create a new branch called "feature-login" off the `develop` branch.
2. The team members switch to the new branch using the command `git checkout feature-login`. They can now make changes to the codebase without affecting the main version of the code.
3. One team member makes changes to a file called "`login.py`" and commits the changes using the command `git commit -am "added new function to login.py"`. Another team member makes changes to a file called "`config.py`" and commits the changes using the command `git commit -am "modified config settings"`.
4. After working on the feature for several days, the team decides that the feature is ready to be merged back into the develop branch. They switch back to the develop branch using the command `git checkout develop`, and then use the command `git merge feature-login` to merge the changes from the feature branch.
5. The development team continues to work on other features and bug fixes. Once they have a sufficient number of features and bug fixes, they create a new branch called `release` off of the `develop` branch and make any final adjustments and fixes before the release.
6. Once the release is ready, the team merges the release branch into both the `main` and `develop` branches. The `main` branch is then tagged with a release number and the `develop` branch continues with ongoing development.
7. If there are any bugs found in the `main` branch that need to be fixed, the team creates a new branch called `hotfix` off of the `main` branch. They make the necessary changes and once they are completed they are merged back into `main`.

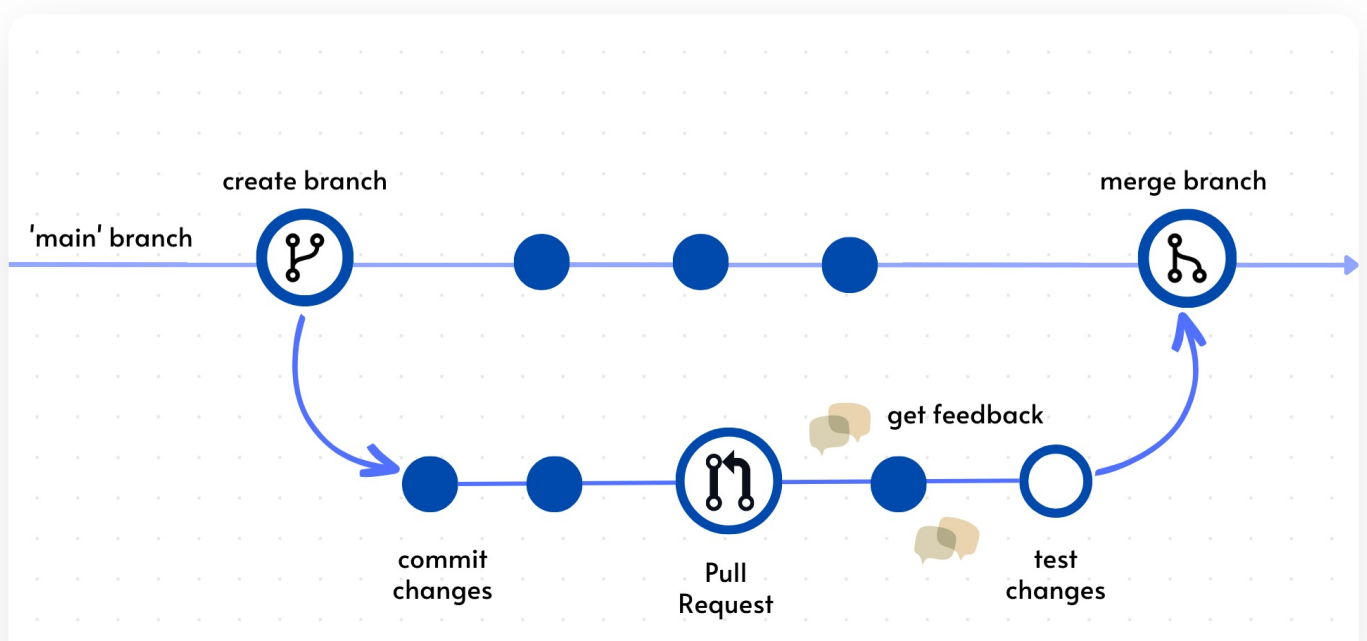
and **develop** branches.

8. After the hotfix is done and merged into **main** branch, the team creates a new release branch off of **develop** branch, and continues to work on new features and bug fixes.
9. This process continues as the team works on new features, releases, and hotfixes. The **main** branch always represents the production-ready code and the **develop** branch is used for ongoing development.

It's important to note that Git flow is just one of many branching models available, and it may not be the best fit for every project. However, it is a widely used and well-established model that can help teams work together effectively and keep the codebase stable and ready for production at all times.

GitHub Workflow

GitHub workflow is a Git-based model that provides a systematic approach for managing software development projects in a collaborative environment. It involves a series of steps for managing tasks, making changes, and testing code.



The following is an overview of the GitHub workflow process:

1. **Create a repository:** Create a new repository for your project on GitHub. This repository will store your code and enable you to collaborate with others.
2. **Create a branch:** Create a new branch from the main branch for each feature or bug fix that you want to work on. The branch allows you to work on changes without affecting the main branch.
3. **Make changes:** Make changes to the code in your branch. Commit your changes frequently to the branch. This keeps track of your changes and enables you to revert to a previous version of the code if needed.
4. **Open a pull request:** When you are ready to merge your changes into the main branch, open a pull request. The pull request shows the changes you made, and it allows other team members to review and comment on your code.
5. **Review and test:** Team members review your changes and test the code to ensure that it works as expected. If there are any issues, they provide feedback, and you make changes as needed.
6. **Merge changes:** Once the code has been reviewed and tested, it can be merged into the main branch. This ensures that all team members are working from the same codebase.
7. **Deploy and monitor:** Once the changes are merged, they can be deployed to the production environment. The code is then monitored to ensure that it is functioning as expected.
8. **Close the pull request:** Once the changes have been merged, the pull request can be closed. This keeps the repository clean and organized.

Overall, the GitHub workflow process enables multiple team members to collaborate on the same codebase, while ensuring that changes are made systematically and with proper review and testing.

Others

- **Origin:**

In Git, `origin` is a default name given to the remote repository that a local repository was originally cloned from. The remote repository name is used to specify the location of the repository on a server and it can be used to interact with the remote repository.

- **HEAD:**

In Git, the `HEAD` is a special reference that points to the most recent commit on the current branch. It is used as a way to refer to the current state of the repository and can be thought of as a pointer to the tip of the current branch.

- **Writing Commit Messages:**

Writing clear and concise commit messages is an important best practice in Git, as it helps you and your collaborators understand the changes that were made and why. Here are some tips for writing effective Git commit messages:

1. Keep it short: Try to keep your commit message under 50 characters, as this is the default length that Git uses to display messages in some contexts.
2. Be descriptive: Your commit message should describe the changes that were made and why they were made. Use clear and specific language to help others understand the purpose of the changes.
3. Use the imperative mood: Write your commit messages in the imperative mood, as if you are giving a command. For example, instead of saying "Fixed a bug", say "Fix bug in login form". This makes the message more action-

oriented and easier to understand.

4. Include relevant information: If your commit relates to an issue or pull request, include the reference number in your message. You can also include links to relevant documentation or resources.
5. Break it up: If your commit includes multiple changes or fixes, break up the message into bullet points to make it easier to read.
6. Proofread: Take the time to review your commit message before submitting it. Typos and unclear language can make it harder for others to understand the purpose of the changes.

Remember, commit messages are a form of documentation, so it's important to take them seriously and put in the effort to make them clear and informative.

📁 Programming

📁 Tutorial



Shubham Sarda

Senior Software Engineer & Instructor



Shubham Sarda is a full-stack developer, coding mentor, and tech educator with years of industry experience. Passionate about making coding simple, he has helped thousands of students master Python, Django, JavaScript, React and full-stack development through hands-on learning.

His teaching approach focuses on breaking down complex concepts into easy-to-follow lessons while ensuring students gain real-world coding skills. Whether you're a beginner or an experienced developer looking to upskill, Shubham's courses provide structured guidance, industry insights, and practical projects.

At Unwired Learning, he's here to help you learn faster, code smarter, and build confidently.



Empowering developers through quality education



Popular Courses

Python Masterclass

JavaScript Masterclass

React Masterclass

Data Structures & Algorithms

Quick Links

[About Us](#)

[Contact](#)

[Testimonials](#)

[Blog](#)

Others

[Verify Certificate](#)

[Terms of Service](#)

[Privacy Policy](#)

[FAQ](#)